

 SHIPMENT TRACKER DASHBOARD

Feature & Technology Presentation for Viva

Project: Shipment Tracker Dashboard
Organization: Wear Well India
Date: March 2026

PAGE 1: PROJECT OVERVIEW

What is This Project?

A **web-based shipment management system** that allows:

- **Users** to create and track their shipments
- **Admins** to manage, approve, and monitor all shipments
- **Real-time updates** when shipment status changes

Business Problem Solved

Problem	Solution
Manual tracking via phone/email	Centralized digital dashboard
No visibility on shipment status	Real-time status updates
No record keeping	Cloud database stores everything
Security concerns	User authentication & role-based access

PAGE 2: TECHNOLOGY STACK

Frontend Technologies

Technology	What It Does
React	Builds the user interface with reusable components
Vite	Fast development server & build tool
Tailwind CSS	Styles the application (colors, layouts, responsive design)

Backend/Cloud Services

Service	What It Does
Firebase Firestore	Cloud database that stores shipments & notifications

Service	What It Does
Clerk	Handles user login, signup, and authentication

Supporting Libraries

Library	What It Does
Lucide React	Provides icons (search, bell, truck icons)
Recharts	Creates analytics charts and graphs

PAGE 3: WHY THESE TECHNOLOGIES?

React - Frontend Framework

Why React over plain HTML/JavaScript?

- **Components:** Build once, reuse everywhere (like building blocks)
- **Automatic Updates:** When data changes, UI updates automatically
- **Industry Standard:** Used by Facebook, Instagram, Netflix, Airbnb

Firebase Firestore - Database

Why Firestore over MySQL/MongoDB?

- **Real-time:** Data syncs instantly (no refresh needed)
- **No Server Required:** Directly connects from browser
- **Auto-scaling:** Handles any amount of data
- **Free Tier:** Good for small projects

Clerk - Authentication

Why Clerk over building our own login?

- **Secure:** Handles passwords, sessions, encryption
- **Pre-built UI:** Login/signup forms ready to use
- **Social Login:** Google, GitHub login with one click
- **Fast Setup:** Minutes instead of weeks

PAGE 4: USER ROLES & PERMISSIONS

Two Types of Users

 Regular User (Customer)

Can Do	Cannot Do
--------	-----------

Can Do	Cannot Do
☒ Create new shipments	☒ View other users' shipments
☒ View their own shipments	☒ Approve/reject shipments
☒ See notifications about their shipments	☒ Access admin dashboard
☒ Search & filter their shipments	☒ Delete shipments

 Admin

Can Do	Special Powers
☒ View ALL shipments from all users	☒ Separate admin login
☒ Approve or reject shipments	☒ Access analytics page
☒ Update shipment status	☒ See all new shipment alerts
☒ Delete shipments	☒ View business statistics

PAGE 5: CORE FEATURES - SHIPMENT MANAGEMENT

Feature 1: Create Shipment

User Action: Fill form with shipment details

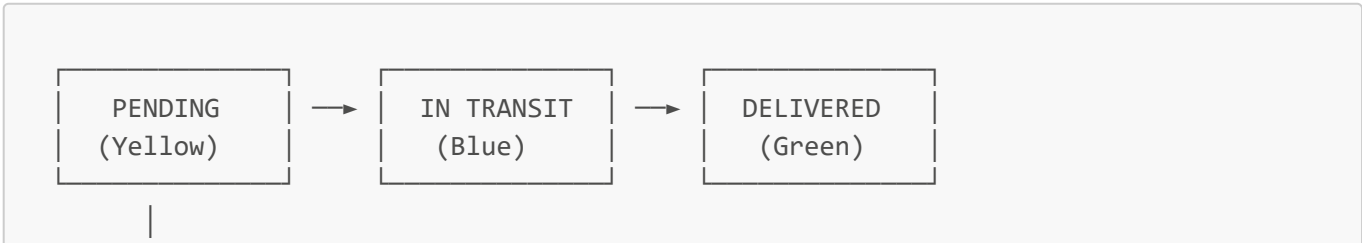
Information Captured:

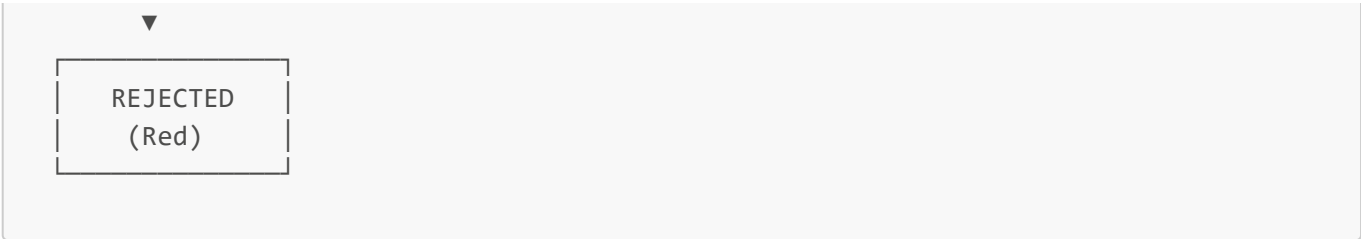
- Tracking Number (auto-generated or manual)
- Customer Name
- Product Name
- Shipment Date
- Destination Address

What Happens Behind the Scenes:

1. Form validates all fields
2. Data sent to Firebase Firestore
3. Admin receives notification
4. User sees shipment in their list

Feature 2: Shipment Status Workflow





Status Meanings:

- **Pending:** Waiting for admin approval
- **In Transit:** Shipment is on the way
- **Delivered:** Successfully delivered
- **Rejected:** Admin declined the shipment

PAGE 6: CORE FEATURES - SEARCH & FILTER

Feature 3: Search Functionality

Users can search by:

- 🔍 Tracking Number
- 👤 Customer Name
- 📦 Product Name

How It Works:

- Type in search box
- Results filter instantly (no button click needed)
- Case-insensitive (finds "JOHN" when you type "john")

Feature 4: Filter by Status

Filter Options:

Filter	Shows
All	Every shipment
Pending	Only pending shipments
In Transit	Only shipments in transit
Delivered	Only delivered shipments

Use Case: Admin wants to see only pending shipments to approve them quickly

Feature 5: Sort Functionality

Sort Options:

- By Shipment Date (newest/oldest first)
- By Customer Name (A-Z / Z-A)
- By Product Name (A-Z / Z-A)
- By Tracking Number

PAGE 7: NOTIFICATION SYSTEM

How Notifications Work

Bell Icon 

- Shows unread count badge
- Click to see notification list
- Mark as read option

User Notifications

Event	Notification Message
Shipment Approved	"Your shipment #12345 has been approved!"
Shipment Rejected	"Your shipment #12345 was rejected"
Status Changed	"Your shipment #12345 is now In Transit"

Admin Notifications

Event	Notification Message
New Shipment	"New shipment #12345 from user@email.com"

Real-Time Updates

- Notifications appear **instantly** without page refresh
- Firebase real-time listeners detect new notifications
- Badge count updates automatically

PAGE 8: AUTHENTICATION SYSTEM

User Authentication (Clerk)

Login Options

-  Email + Password
-  Google Account
-  GitHub Account

Security Features

Feature	Description
Session Management	Stay logged in across browser tabs
Secure Tokens	Encrypted authentication tokens
Auto Logout	Session expires for security
Password Reset	Email-based password recovery

Admin Authentication (Custom)

Separate Admin Portal

- Different login page for admins
- Username + Password based
- No social login (more controlled access)

Why Separate Admin Auth?

- Admins need different permissions
- More control over who becomes admin
- Separate session management

PAGE 9: USER INTERFACE FEATURES

Dark/Light Theme Toggle

How It Works

- 🌙 Dark Mode: Easy on eyes, modern look
- ☀️ Light Mode: Traditional bright interface
- **Remembers Choice**: Saves preference in browser storage

Technical Implementation

- Theme state stored in Context (global state)
- localStorage persists choice across sessions
- Tailwind CSS classes switch based on theme

Responsive Design

Works On All Devices

Device	Layout
--------	--------

Device	Layout
 Desktop	Full table view with all columns
 Mobile	Stacked cards, touch-friendly buttons
 Tablet	Adaptive layout

Tailwind CSS Breakpoints

- Mobile First approach
- Automatically adjusts at different screen sizes

PAGE 10: DATABASE STRUCTURE

Firestore Collections

Collection: **shipments**

Field	Type	Description
trackingNumber	String	Unique shipment identifier
customerName	String	Customer's name
productName	String	Product being shipped
shipmentDate	Timestamp	Date of shipment
status	String	Current status
userEmail	String	Who created it
createdAt	Timestamp	When created
updatedAt	Timestamp	Last modified

Collection: **notifications**

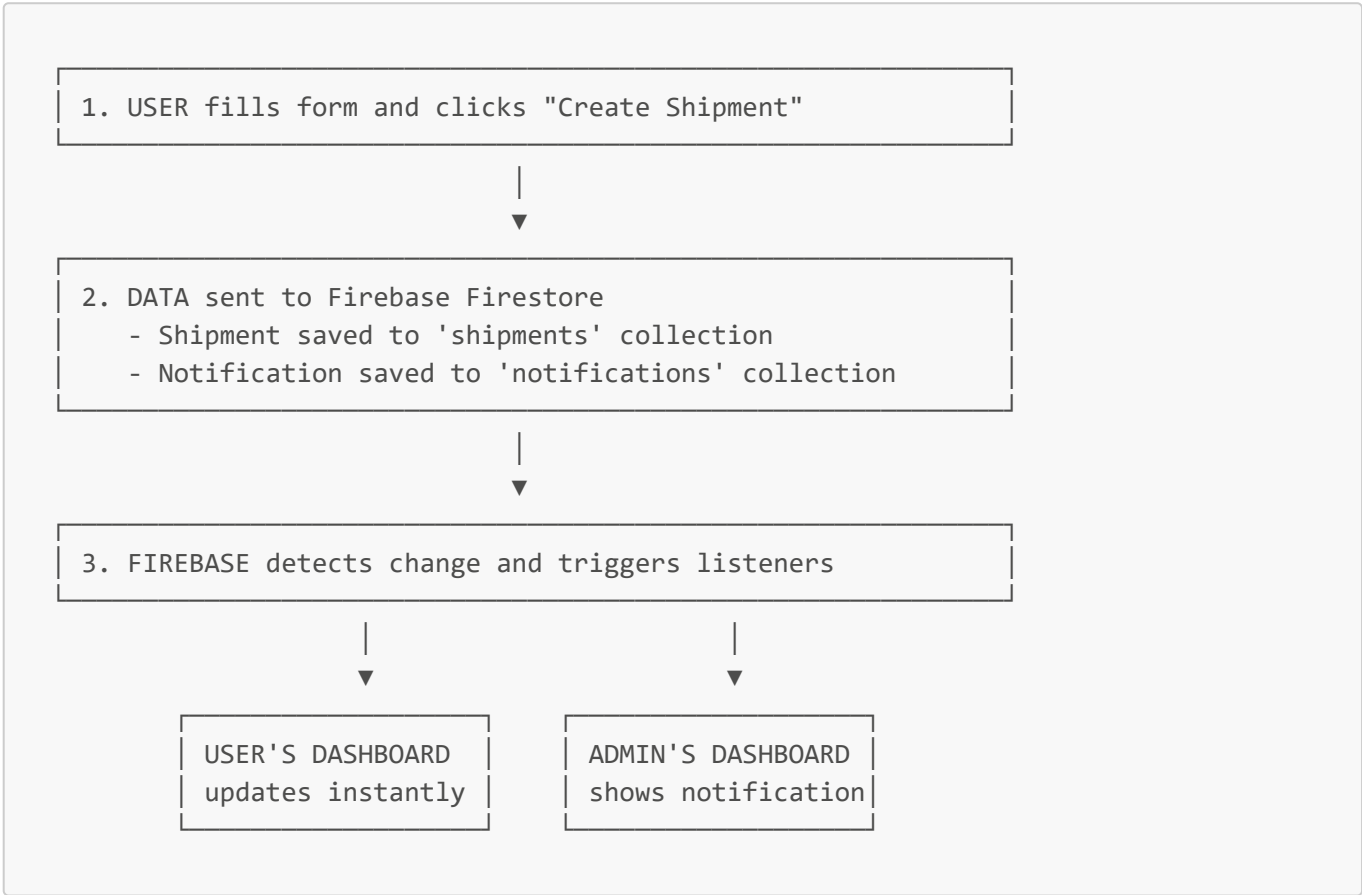
Field	Type	Description
type	String	"new_shipment" or "status_change"
recipientType	String	"admin" or "user"
recipientEmail	String	User's email (for user notifications)
trackingNumber	String	Related shipment
message	String	Notification text
read	Boolean	Has been read?

Field	Type	Description
createdAt	Timestamp	When created

PAGE 11: REAL-TIME DATA FLOW

How Real-Time Updates Work

Scenario: User Creates Shipment



Key Concept: Firestore Listeners

- Application "subscribes" to data changes
- Firebase pushes updates (no polling needed)
- UI updates automatically without refresh

PAGE 12: ANALYTICS DASHBOARD

Feature: Business Analytics

Charts & Statistics Available

Chart Type	Shows
------------	-------

Chart Type	Shows
Pie Chart	Shipments by status (Pending, In Transit, Delivered)
Bar Chart	Shipments per day/week
Line Chart	Trend over time
Summary Cards	Total shipments, pending count, delivered count

How Charts Are Built

- **Recharts Library:** React-friendly charting
- Data fetched from Firestore
- Aggregated and formatted for visualization
- Responsive design (works on mobile)

PAGE 13: SECURITY MEASURES

Data Security

Layer	Protection
Authentication	Only logged-in users can access
Authorization	Users only see their own data
Firebase Rules	Database-level security rules
HTTPS	Encrypted data transmission

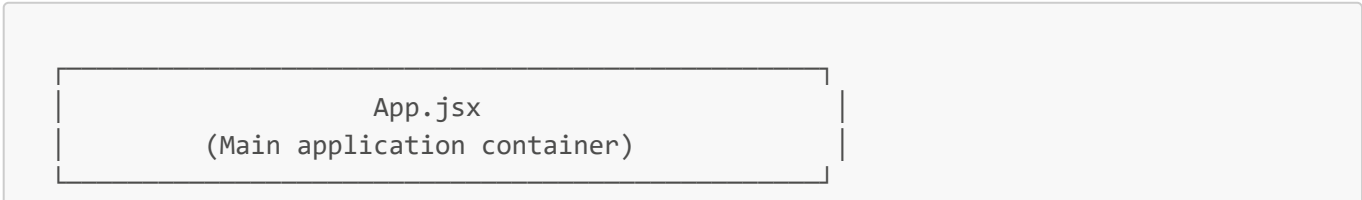
User Data Isolation

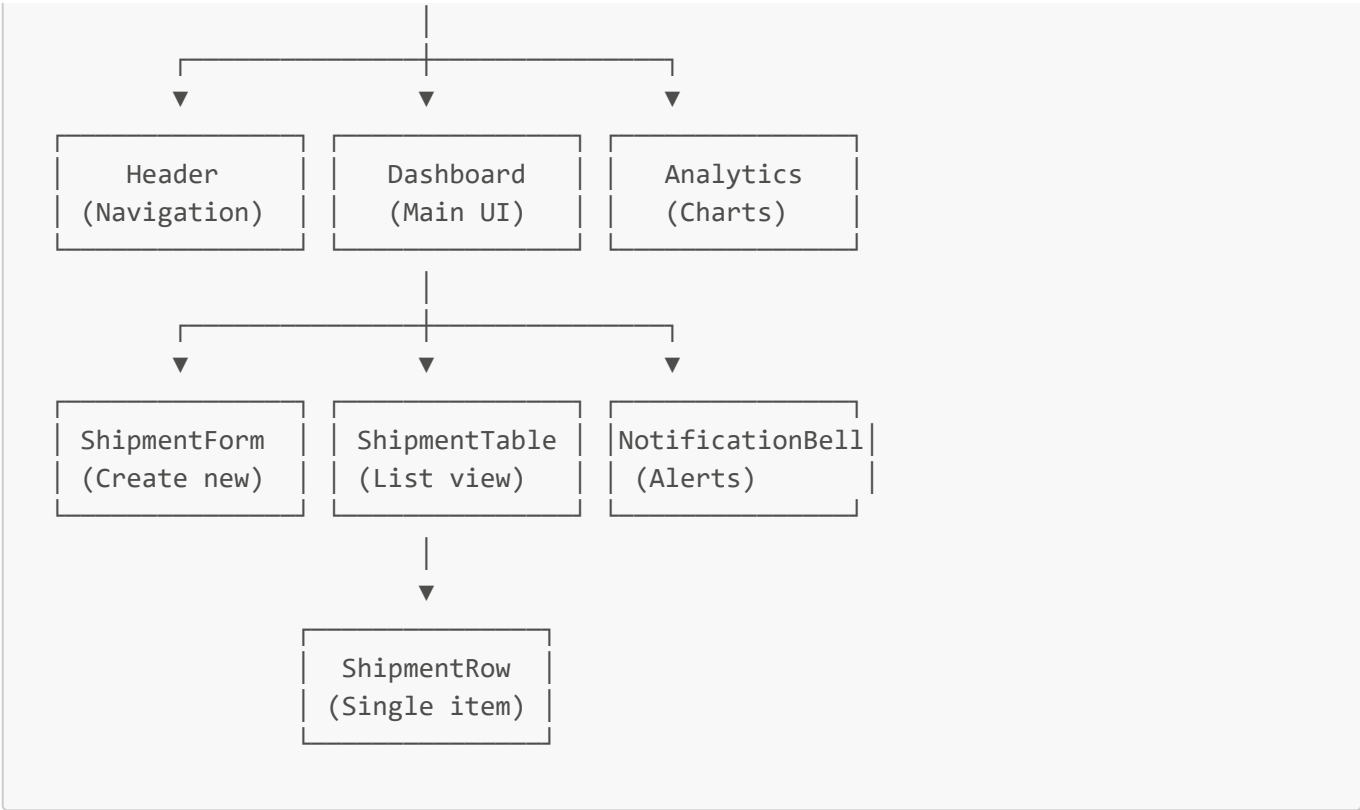
How User A Cannot See User B's Shipments

1. Every shipment stores `userEmail` field
2. Query filters: `WHERE userEmail == currentUser.email`
3. Firebase security rules enforce this at database level
4. Even if someone hacks frontend, database rejects unauthorized requests

PAGE 14: APPLICATION ARCHITECTURE

Component Hierarchy





State Management

State	Where Stored	Why
Theme (dark/light)	ThemeContext	Needed by all components
Admin login status	AuthContext	Needed for routing
User session	Clerk	Managed by Clerk SDK
Shipments list	App.jsx useState	Parent manages, passes to children

PAGE 15: KEY TECHNICAL CONCEPTS FOR VIVA

React Concepts Used

Concept	Where Used	Purpose
Components	Every file	Reusable UI pieces
useState	Forms, lists	Store changing data
useEffect	Data fetching	Run code after render
useContext	Theme, Auth	Share data globally
Props	Parent to child	Pass data down
Conditional Rendering	Admin vs User view	Show different UI based on condition

Firebase Concepts Used

Concept	Where Used	Purpose
addDoc	Create shipment	Add new document
onSnapshot	Real-time list	Listen for changes
query + where	Filter shipments	Get specific documents
updateDoc	Status change	Modify existing document
deleteDoc	Remove shipment	Delete document
serverTimestamp	Created/Updated times	Server-generated timestamps

Tailwind CSS Concepts Used

Concept	Example	Purpose
Utility Classes	bg-blue-500, p-4	Quick styling
Responsive	md:flex, lg:grid	Different layouts per screen
Dark Mode	dark:bg-gray-800	Theme-based colors
Hover States	hover:bg-blue-600	Interactive feedback

QUICK REFERENCE SUMMARY

Project At a Glance

Aspect	Details
Frontend	React + Vite + Tailwind CSS
Database	Firebase Firestore (NoSQL, Real-time)
Auth	Clerk (Users) + Custom (Admin)
Features	CRUD, Search, Filter, Sort, Notifications, Analytics
Users	Regular Users + Admin
Real-time	Yes (Firebase listeners)
Responsive	Yes (Mobile + Desktop)
Theme	Dark + Light mode

5 Things to Remember

- 1. **React** = Component-based UI library
- 2. **Firebase** = Real-time cloud database (no backend server needed)
- 3. **Clerk** = Pre-built authentication service
- 4. **Tailwind** = Utility-first CSS framework

5. **Real-time** = Data updates without page refresh using Firestore listeners

End of Presentation